



Politecnico di Milano

N-Body Problem

Serial and Parallel Solvers for the N-Body Simulation Problem

By

Di Paola Giuseppe
Boudarraja Karim
Nuttini Elena

Professor: Formaggia Luca

Exam Project

High Performance Computing Engineering

Settembre 2026

Contents

1	Introduction	3
1.1	The N-body problem	3
1.2	Structure of this report	3
2	Force Models	4
2.1	The Force class hierarchy	4
2.2	Shared computation and numerical safeguards	4
2.3	Implemented force laws	5
3	Numerical Integration	6
3.1	Explicit Euler integration	6
3.2	Velocity Verlet integration	6
3.3	Choice of integrator per algorithm	7
4	The Standard (Brute-Force) Algorithm	8
4.1	Serial implementation and Newton's third law	8
4.2	Complexity	8
5	Collision Handling	10
5.1	Boundary collisions	10
5.2	Pairwise collisions via spatial hashing	10
5.3	The elastic collision model	11
6	Parallel Standard Algorithm	12
6.1	Overview	12
6.2	The local-forces reduction pattern	12
6.3	Scheduling choices	13
6.4	Efficiency considerations	13
7	The Barnes-Hut Algorithm (Serial)	14
7.1	The QuadtreeNode data structure and its ownership model	14
7.2	Tree construction	14
7.3	Force evaluation	15
7.4	Efficiency considerations	16
8	Parallel Barnes-Hut Algorithm	17
8.1	Domain decomposition and border assignment	17
8.2	Parallel subtree construction	18
8.3	Task-based merging	18
8.4	Efficiency considerations	19

9	File Handling	20
9.1	Info.txt	20
9.2	Coordinate files and the speedUp parameter	20
9.3	Thread and file count	20
10	Performance Evaluation	22
10.1	Methodology	22
10.2	Serial brute-force vs. serial Barnes-Hut	22
10.3	Parallel speedup at 2 threads	23
10.4	Approximation error of Barnes-Hut	24
10.4.1	Methodology	24
10.4.2	Error vs. theta	24
10.4.3	Error vs. number of subtrees	25
10.4.4	Error under Verlet-integrated dynamics	26
11	Conclusions and Future Work	27
11.1	Possible extensions	27

Chapter 1

Introduction

1.1 The N-body problem

The N-body problem consists of predicting the trajectories of a set of particles that interact with one another through a pairwise force. The most familiar instance is gravitational: the motion of planets, stars, or galaxies under Newton’s law of universal gravitation. The same mathematical structure, however, applies to any distance-dependent interaction, including the electrostatic attraction and repulsion between charged particles described by Coulomb’s law, which this project also implements.

For a system of n particles, each particle’s equation of motion depends on the position of every other particle, coupling all n second-order differential equations together. Closed-form analytical solutions exist only for very small systems (the two-body problem, and special configurations of the three-body problem); beyond that, numerical integration is the only practical approach. Two computational challenges follow directly from this: first, how to evaluate, at every time step, the net force acting on every particle; second, how to advance positions and velocities forward in time from those forces without accumulating unacceptable numerical error.

This report addresses both challenges. The force-evaluation problem is solved with two algorithms of different asymptotic cost — an exact $O(n^2)$ pairwise evaluation and an approximate $O(n \log n)$ hierarchical evaluation (Barnes-Hut) — each available in a serial and an OpenMP-parallel variant. The time-integration problem is solved with two schemes of different accuracy: explicit Euler and Velocity Verlet.

1.2 Structure of this report

Chapter 2 describes the force-model hierarchy and the numerical safeguards shared by all force laws. Chapter 3 covers the two time-integration schemes. Chapter 4 introduces the brute-force algorithm and its use of Newton’s third law. Chapter 5 describes collision handling, which in this implementation is a separate subsystem shared by all four simulation modes. Chapters 6 through 8 cover, respectively, the parallel brute-force algorithm, the serial Barnes-Hut algorithm, and the parallel Barnes-Hut algorithm, including the quadtree’s ownership model and its task-based parallel merge. Chapter 9 covers file output. Chapter 10 presents measured performance results, including parallel speedup. Chapter 11 summarizes the work and lists extensions that were considered but are not part of the current implementation.

Chapter 2

Force Models

2.1 The Force class hierarchy

Every pairwise interaction law in the project derives from an abstract base class, `Force<Dimension>`, whose only public responsibility is a pure virtual method, `calculateForce`, that returns the force vector exerted by one particle on another:

```
template<size_t Dimension>
class Force {
public:
    virtual ~Force() = default;
    virtual std::array<double, Dimension> calculateForce(
        const Particle<Dimension> &p1, const Particle<Dimension> &p2) const = 0;
};
```

This is a Strategy design pattern: every consumer of a force object — the brute-force loop, the Barnes-Hut traversal — holds only a reference to `Force<Dimension>`, and is therefore fully decoupled from which concrete force law is in effect. Adding a new force to the simulation never requires modifying the algorithms that use it.

2.2 Shared computation and numerical safeguards

Every concrete force law needs the same three geometric quantities: the displacement vector between the two particles, its squared magnitude, and the cubed magnitude used to normalize the direction. These are factored into protected static helpers on the base class rather than duplicated in every subclass:

```
protected:
    static std::array<double, Dimension> calculateDisplacement(
        const Particle<Dimension> &first, const Particle<Dimension> &second);
    static double calculateDistanceSquared(
        const std::array<double, Dimension> &displacement);
    static double distanceCubed(double distanceSquared);
    static std::array<double, Dimension> assembleForce(
        const std::array<double, Dimension> &displacement, double factor);
```

The most important of these for numerical robustness is `calculateDistanceSquared`, which clamps its result to a minimum of 10^{-24} instead of letting it reach exactly zero:

```
constexpr double minimumDistanceSquared = 1e-24;
return distanceSquared < minimumDistanceSquared
    ? minimumDistanceSquared
    : distanceSquared;
```

Without this floor, two particles occupying, or very nearly occupying, the same position would produce a division by a value arbitrarily close to zero in every force law, propagating `Inf` or `NaN` silently through the rest of the simulation. Clamping the squared distance is a standard softening technique in N-body codes; applying it once, in a shared helper, guarantees that every force law benefits from it uniformly instead of relying on each subclass to reimplement the safeguard correctly.

2.3 Implemented force laws

Four concrete force laws are provided:

- **GravitationalForce** — Newton’s law of universal gravitation, $F = Gm_1m_2/r^2$, with $G = 6.674 \times 10^{-11}$. Both particle properties (masses) are always positive, so the resulting force is always attractive.
- **CoulombForce** — Coulomb’s law, $F = Kq_1q_2/r^2$, with $K = 8.987 \times 10^{-9}$. Particle properties here represent electric charge and may be positive or negative, so the force law includes an explicit sign flip relative to the gravitational case, producing repulsion between like charges.
- **RepulsiveForce** — a short-range repulsive force proportional to the inverse fourth power of distance, used as a generic separation force independent of particle mass or charge.
- **CustomForce** — structurally identical to **GravitationalForce**, but with the coupling constant supplied at construction time rather than hard-coded, intended for controlled test scenarios such as the two-body orbit test described in Chapter 2.

Each concrete class implements only `calculateForce`, expressed in terms of the four shared helpers; the physics-specific content of each class reduces to the sign and the exponent of the force law, which keeps the implementations easy to compare and to audit for correctness.

Chapter 3

Numerical Integration

Two integration schemes are implemented on the `Particle` class: an explicit Euler update and a Velocity Verlet update. Both operate on the same three quantities stored per particle: position, velocity, and the net force accumulated during the current step.

3.1 Explicit Euler integration

The Euler update advances position and velocity using the values available at the start of the step:

```
void update(const double delta_t) {
    for (size_t i = 0; i < Dimension; ++i) {
        pos[i] += vel[i] * delta_t;
        vel[i] += (force[i] / ((property < 0) ? -property : property)) * delta_t;
    }
}
```

The division by the absolute value of `property`, rather than by `property` directly, is what allows a single `Particle` class to serve both the gravitational case (`property` is a mass, always non-negative) and the Coulomb case (`property` is a charge, which may be negative): the inertial response of a particle to a force must always use its mass, i.e. the magnitude of the stored `property`, never its signed value.

This is the explicit (forward) Euler method: the position update uses the velocity from the beginning of the step, and the velocity update uses the force from the beginning of the step; neither uses a value produced later in the same step. This ordering is not symplectic, and therefore does not conserve the total mechanical energy of the system over long integration runs — the energy of a Kepler orbit integrated this way tends to drift monotonically rather than oscillate around its true value, and larger time steps make the drift worse. The method remains useful for short simulations or for scenarios where exact energy conservation is not the primary concern, and it costs no more per step than the Velocity Verlet alternative described next.

3.2 Velocity Verlet integration

The second integration scheme reuses the acceleration computed in the previous step to obtain second-order accuracy:

```
void velocityVerletUpdate(const double delta_t) {
    std::array<double, Dimension> prevAccel;
    for (size_t i = 0; i < Dimension; ++i) {
        prevAccel[i] = accel[i];
        accel[i] = (force[i] / ((property < 0) ? -property : property));
        pos[i] += vel[i] * delta_t + 0.5 * prevAccel[i] * delta_t * delta_t;
    }
}
```

```
        vel[i] += 0.5 * (prevAccel[i] + accel[i]) * delta_t;
    }
}
```

This is the position form of the Velocity Verlet algorithm: the new position uses both the current velocity and the previous step’s acceleration (a standard second-order Taylor expansion of position), while the new velocity is updated using the average of the previous and current accelerations. Velocity Verlet is a symplectic integrator: for conservative forces it does not drift in total energy over arbitrarily long integration times, only oscillates around the true value with an amplitude that shrinks as the time step shrinks. This property, together with its identical per-step cost to explicit Euler, is why `velocityVerletUpdate` is the integrator used by both the serial and parallel Barnes-Hut implementations (Chapters 7 and 8), while the brute-force implementations (Chapters 4 and 6) use the plain Euler update.

3.3 Choice of integrator per algorithm

The two brute-force algorithms (serial and parallel) call `update(delta_t)`; the two Barnes-Hut algorithms call `velocityVerletUpdate(delta_t)`. This is a structural asymmetry worth flagging explicitly: the two families of algorithms currently produce trajectories of different numerical quality for the same physical system, and a direct trajectory-accuracy comparison between, say, serial brute-force and serial Barnes-Hut should account for this difference rather than attribute all of it to the force approximation. Unifying the integrator across all four modes is listed as a possible extension in Chapter 11.

Chapter 4

The Standard (Brute-Force) Algorithm

4.1 Serial implementation and Newton’s third law

The serial brute-force algorithm, `serialSimulation`, evaluates the exact force between every pair of particles once per time step. For each particle `i`, the inner loop only considers particles `j > i`:

```
for (std::size_t i = 0; i < particles->size(); ++i) {
    Particle<Dimension>& q = (*particles)[i];
    q.manageCollision(dim);
    for (std::size_t j = i + 1; j < particles->size(); j++) {
        Particle<Dimension>& k = (*particles)[j];
        q.manageCollision(k, softening);
        force_qk = f.calculateForce(q, k);
        q.addForce(force_qk);
        for (size_t i = 0; i < Dimension; ++i) force_qk[i] = -force_qk[i];
        k.addForce(force_qk);
    }
    z == it - 1 ? q.update(delta_t) : q.updateAndReset(delta_t);
}
```

This is the standard application of Newton’s third law to a pairwise force: the force exerted by `k` on `q` is computed once, added to `q`, and the negated vector is added to `k`, instead of evaluating `calculateForce` a second time with the arguments reversed. This halves the number of calls to `calculateForce`, which is the dominant cost of the loop, without changing the physical result.

The alternating call at the end of the outer loop, `q.update(delta_t)` on the final iteration and `q.updateAndReset(delta_t)` otherwise, is a minor optimization: `updateAndReset` performs the same Euler update as `update` and additionally zeroes the accumulated force in preparation for the next step; on the very last iteration there is no next step, so the reset is skipped.

Note that collision handling is performed inline, in the same loop that computes forces, rather than through the spatial-hash mechanism described in Chapter 5. This is a deliberate choice specific to this algorithm: because the force loop already visits every pair of particles at $O(n^2)$ cost, checking for a collision on the same pair adds no additional asymptotic cost. The spatial hash exists to avoid an $O(n^2)$ neighbor search where one would otherwise be introduced on top of a cheaper, non-quadratic force evaluation — which is exactly the situation in the parallel brute-force algorithm and in both Barnes-Hut algorithms, described in the following chapters.

4.2 Complexity

For n particles, the double loop performs $n(n-1)/2$ force evaluations per time step, giving $O(n^2)$ complexity per step and $O(n^2 \cdot T)$ for a simulation of T steps. This is the accuracy baseline

against which the Barnes-Hut approximation in Chapters 7 and 8 is compared: every force in this algorithm is exact, with no angular approximation.

Chapter 5

Collision Handling

Three of the four simulation modes — `parallelSimulation`, `serialBarnesHut`, and `parallelBarnesHut` — resolve particle collisions through two dedicated functions, `resolveBoundaryCollisions` and `resolveCollisions`, called once per time step before force computation begins. (`serialSimulation` handles collisions inline, for the reason explained in Section 4.1.) This chapter describes both functions and the elastic collision model they invoke.

5.1 Boundary collisions

`resolveBoundaryCollisions` is the simpler of the two: each particle independently checks whether it has crossed the simulation boundary and, if so, reflects its velocity component along the violated axis. Because every particle only ever reads and writes its own state, the loop parallelizes with a plain static schedule and no further synchronization:

```
template <size_t Dimension>
void resolveBoundaryCollisions(std::vector<Particle<Dimension>>& particles, double boundary)
{
    #pragma omp parallel for schedule(static)
    for (std::size_t i = 0; i < particles.size(); ++i) {
        particles[i].manageCollision(boundary);
    }
}
```

5.2 Pairwise collisions via spatial hashing

Detecting collisions between particles is, in principle, an all-pairs problem like force evaluation. `resolveCollisions` avoids paying that cost separately from the (already handled, and in the Barnes-Hut case sub-quadratic) force evaluation by first bucketing every particle into a uniform grid of square cells, then only comparing each particle against the contents of its own cell and the eight neighboring cells.

Cell sizing and the two-phase detect/apply pattern

```
const double cellSize = 2.0 * maximumRadius * std::sqrt(softening);
```

`maximumRadius` is the largest particle radius present in the simulation. Sizing the cell at twice that radius (scaled by the collision-tolerance factor `softening`) guarantees that any two particles close enough to collide can never be more than one cell apart on either axis — which is exactly the property that makes a 3×3 neighborhood search sufficient and correct, regardless of how particles happen to be distributed across cells.

Particles are first assigned to cells using an `std::unordered_map` keyed on integer cell coordinates. Detection then proceeds in two distinct phases:

- **Phase 1 — parallel detection.** Each thread processes a disjoint range of particles (`schedule(static)`); for each particle it searches only the 3×3 block of cells centered on the particle's own cell, and records every colliding pair it finds — checking only $j > i$ to avoid recording the same pair twice — into a thread-local buffer, `localCollisions[threadId]`. No particle is ever mutated during this phase.
- **Phase 2 — serial application.** After the parallel region ends, the collisions recorded by every thread are applied sequentially, one call to `manageCollision` per pair.

The second phase must be serial because `manageCollision` mutates the velocities of both particles in a pair simultaneously (see Section 5.3). If two threads discovered two different collisions that happen to share one particle — for instance, collisions (i, j) and (j, k) found concurrently on different threads — applying both at the same time without synchronization would be a data race on particle j 's velocity. Separating detection (embarrassingly parallel, read-only) from application (small in volume relative to detection, and sequential) sidesteps this race entirely rather than introducing locks or atomics around every collision.

5.3 The elastic collision model

`manageCollision(Particle&, softening)` implements a physically motivated elastic collision between two particles of masses equal to the absolute value of their stored property (so that the model remains meaningful for both the gravitational case, where property is a mass, and the Coulomb case, where property is a signed charge). The implementation computes the unit normal between the two particle centers, the relative velocity along that normal, and — only if the particles are approaching rather than separating — an impulse that conserves both momentum and kinetic energy along the collision normal:

```
const double impulse = -2.0 * relativeNormalVelocity /
    (1.0 / firstMass + 1.0 / secondMass);
```

This is the standard one-dimensional elastic collision formula applied along the collision normal, generalized to `Dimension` coordinates. The guard on `relativeNormalVelocity` ensures that a pair of particles that are already moving apart — which can legitimately still be found by the 3×3 spatial search as it does not distinguish approach from separation — is left untouched rather than having an unphysical impulse applied to it.

Chapter 6

Parallel Standard Algorithm

6.1 Overview

`parallelSimulation` parallelizes the same $O(n^2)$ force evaluation as Chapter 4 with OpenMP, using a per-thread local accumulator to avoid concurrent writes to a shared force array, and delegates collision handling to `resolveBoundaryCollisions` and `resolveCollisions` (Chapter 5) rather than performing it inline. Each time step proceeds through four phases inside a single parallel region: force accumulation, reduction of the per-thread accumulators into each particle, position/velocity update, and — periodically — output.

6.2 The local-forces reduction pattern

Two threads that each own a different pair of particles can still both need to write to the same particle — for example, thread A computing the pair (2,5) and thread B computing the pair (2,9) both contribute to particle 2’s net force. Writing directly to a shared `Particle` would be a race. The algorithm avoids this with a three-dimensional buffer, `local_forces`, indexed by `[thread][particle][coordinate]`: every thread accumulates its own contributions in its own slice, with no synchronization needed during the accumulation phase itself.

```
#pragma omp for schedule(dynamic, chunkSize)
for (std::size_t i = 0; i < num_particles; ++i) {
    k = &(*particles)[i];
    for (std::size_t j = i + 1; j < num_particles; ++j) {
        q = &(*particles)[j];
        force = f.calculateForce(*k, *q);
        for (std::size_t y = 0; y < Dimension; ++y) {
            local_forces[id_thread][i][y] += force[y];
            local_forces[id_thread][j][y] -= force[y];
        }
    }
}
```

Newton’s third law is still exploited here exactly as in the serial case (line `local_forces[id_thread][j][y] -= force[y]`), so the halving of force evaluations is preserved under parallelization.

Once every thread has finished accumulating, a second loop sums the per-thread contributions into each particle and resets its force to that sum:

```
#pragma omp for schedule(static, particles->size()/omp_get_num_threads())
for (std::size_t i = 0; i < num_particles; ++i) {
    q = &(*particles)[i];
    q->resetForce();
    for (int j = 0; j < num_threads; ++j) {
```

```

        q->addForce(local_forces[j][i]);
    }
}

```

Each thread then clears only its own slice of `local_forces` in preparation for the next time step, followed by an explicit barrier before the position update begins — this keeps the reset phase local to each thread’s own cache lines rather than having one thread clear data that other threads wrote.

6.3 Scheduling choices

The force-accumulation loop uses dynamic scheduling with a configurable chunk size (`schedule(dynamic, chunkSize)`, where `chunkSize` is a command-line parameter, `-chunk`). This is necessary because the inner loop only visits `j > i`: particle 0 requires $n - 1$ inner iterations, particle $n - 1$ requires none, so a static split of the outer loop across threads would give the thread handling the earliest particles far more work than the thread handling the last ones. Dynamic scheduling redistributes chunks of work to threads as they become free, correcting this imbalance. The remaining loops in the time step — the reduction, the reset, the position update, and the output — visit each particle exactly once with uniform cost, so they use static scheduling, which has lower overhead than dynamic scheduling when the workload is already balanced.

6.4 Efficiency considerations

The algorithm’s asymptotic complexity remains $O(n^2)$ per step, unchanged from the serial version; parallelization reduces the constant factor by distributing the $n(n - 1)/2$ force evaluations across the available threads, with an expected speedup approaching the number of threads used as n grows large relative to the per-thread scheduling and reduction overhead. Chapter 10 reports execution-time measurements that characterize this behavior on the hardware available for this report.

Chapter 7

The Barnes-Hut Algorithm (Serial)

The Barnes-Hut algorithm reduces the cost of force evaluation from $O(n^2)$ to $O(n \log n)$ by grouping distant particles into a single approximate body instead of evaluating every pairwise interaction exactly. The grouping is provided by a quadtree: a hierarchical decomposition of the simulation domain in which each node covers a square region, and a region is subdivided into four quadrants only when it needs to distinguish between particles that are too close together to be treated as one aggregate.

7.1 The QuadtreeNode data structure and its ownership model

Each `QuadtreeNode<Dimension>` owns its four children through `std::unique_ptr`, giving the tree the automatic, exception-safe memory management and exclusive-ownership semantics that `unique_ptr` provides for any recursive tree structure. The particles referenced by a node, however, are stored differently: a leaf holds a raw, non-owning pointer to a single particle, and an overflow vector of raw pointers for particles retained past the recursion limit (Section 7.2), rather than `shared_ptr` copies:

```
private:
    const Particle<Dimension>* particle = nullptr;
    std::vector<const Particle<Dimension>*> additionalParticles;
    std::array<std::unique_ptr<QuadtreeNode<Dimension>>, 4> children;
    // ...
```

This choice is deliberate and is documented directly on the class: every particle referenced by the tree lives inside the simulation's own particle vector, which is never resized while a tree built from it is in use, so a tree node can safely reference a particle by address rather than by holding a copy of it. The alternative — giving each node a `shared_ptr<Particle<Dimension>>`, copied from the simulation's particle vector at insertion time — was considered and rejected for a concrete performance reason: it would require one heap allocation and one atomic reference-count increment per particle for every tree built, and a tree is rebuilt from scratch at every simulation step (Section 7.2). Since the particles already live contiguously in the source vector, referencing them by address removes both costs entirely, at the price of an explicit, documented lifetime contract: the class carries a warning stating that the source vector must remain valid and unchanged in size for the entire lifetime of any tree built from it, a contract that both `createQuadTree` (Section 7.2) and the parallel construction path (Chapter 8) satisfy by construction.

7.2 Tree construction

`createQuadTree` builds the tree from scratch at the start of every simulation step, since the particles have moved since the previous step and the previous tree no longer reflects their positions.

The root is sized to cover the entire simulation domain, and every particle is inserted in turn:

```
for (const auto& particle : particles) {
    root->insert(&particle);
}
```

`insert` descends the tree following the same logic in every recursive call: an empty leaf simply stores the incoming particle; a leaf that already holds one particle is split into four children and both particles are re-inserted into the appropriate child; an internal node forwards the particle to the child quadrant matching its position. Every insertion also updates the node's aggregate mass and mass-weighted center of mass incrementally, so that these quantities are available without a second traversal once the tree is complete.

Two of the node's private fields exist specifically to bound the depth of this recursion: `maxDepth` and a numeric width threshold. If a leaf already contains a particle and either the recursion has reached `maxDepth` or the node's width has shrunk to (numerically) zero — both situations that arise only when two or more particles occupy the same, or an extremely close, position — the incoming particle is appended to the leaf's `additionalParticles` overflow list instead of triggering another split. Without this safeguard, coincident particles would cause `split()` to be invoked indefinitely, since halving an already-zero width never produces two particles that fall into different quadrants.

7.3 Force evaluation

`calculateNetForceQuaddtree` computes the net force on a single particle `p` by recursively walking the tree from the root. Three cases are handled at each node:

- A leaf containing a different particle: the exact pairwise force is computed against that particle, and against every particle in its overflow list, if any.
- A leaf with no directly-stored particle (only overflow particles, reached once a node has been driven past its overflow threshold): the exact force is computed against every particle in the overflow list.
- An internal node: the algorithm compares the width of the node's region, s , against its distance from `p`, and decides whether to approximate the node as a single aggregate body or to recurse into its four children.

The approximation test is expressed without an explicit division or square root, for efficiency:

```
if (!node->contains(p->getPos()) && distanceSquared > 0 &&
    s * s < theta * theta * distanceSquared) {
    force_qk = f.calculateForce(*p, approxParticle);
    p->addForce(force_qk);
}
```

$s^2 < \theta^2 d^2$ is algebraically equivalent to the textbook criterion $s/d < \theta$, but avoids computing a square root for every internal node visited during every force evaluation. The `node->contains(p->getPos())` check guards against self-force: if the particle being evaluated happens to lie inside the region represented by the node currently under consideration, the aggregate approximation is skipped even if the width-to-distance ratio would otherwise allow it, and the traversal instead descends into the children — this prevents a particle from being approximated together with, and therefore attracted toward, a center of mass that includes itself.

7.4 Efficiency considerations

Because each level of the quadtree subdivides the region it covers into four equal quadrants, the depth needed to isolate n particles into individual leaves grows only logarithmically with n in a reasonably uniform particle distribution, while regions of higher density are subdivided further, automatically concentrating tree depth — and therefore computational effort — where the particle distribution actually requires it. Evaluating the net force on one particle therefore costs $O(\log n)$ on average, and evaluating it for all n particles costs $O(n \log n)$ per step, against the $O(n^2)$ of the brute-force approach in Chapter 4. Chapter 10 presents execution-time measurements that confirm this asymptotic difference directly from the implementation.

Chapter 8

Parallel Barnes-Hut Algorithm

Building a single quadtree for the whole simulation domain is an inherently sequential process: every insertion depends on the current state of the tree. The parallel Barnes-Hut algorithm sidesteps this by partitioning the simulation domain into independent subregions, building one subtree per subregion concurrently, and then merging the independent subtrees into a single tree. This chapter covers each of the three phases, `generateTreeParallel` being the function that orchestrates all of them.

8.1 Domain decomposition and border assignment

The number of subregions, `num_quad`, is fixed to the smallest power of four not smaller than the number of available threads, so that the domain can always be split into an even grid and merged in groups of four (Section 8.3). Each particle must be assigned to exactly one subregion; a naive half-open interval test on each axis is not sufficient on its own, because a particle sitting exactly on the boundary shared by two subregions needs a rule that assigns it to precisely one of them, not to both and not to neither.

`createQuadTreeParallel` resolves this with four branches that all share the same underlying rule: the lower bound on each axis (`minX`) and the upper bound on the Y axis (`maxY`) are always treated as inclusive, while the upper bound on X (`maxX`) and the lower bound on Y (`minY`) are inclusive only for the subregion that touches the corresponding outer edge of the whole simulation domain, and exclusive otherwise:

```
if (maxX == dimSimulationArea / 2 && minY == -dimSimulationArea / 2) {
    // corner subregion: both maxX and minY inclusive
} else if (minY == -dimSimulationArea / 2) {
    // bottom-row subregion: minY inclusive, maxX exclusive
} else if (maxX == dimSimulationArea / 2) {
    // right-column subregion: maxX inclusive, minY exclusive
} else {
    // interior subregion: both exclusive
}
```

The effect is a tiling in which every subregion is a half-open interval — $[minX, maxX) \times (minY, maxY]$ — that only becomes closed on the sides that coincide with the outer boundary of the whole domain, where there is no neighboring subregion to claim the boundary point instead. This guarantees that every particle, boundary points included, is inserted into exactly one subtree.

8.2 Parallel subtree construction

Once the subregions are determined, `generateTreeParallel` builds one subtree per subregion inside a single parallel for loop, each iteration calling `createQuadTreeParallel` with a different subregion. Since every subtree only reads from the shared particle vector and writes to its own entry of the `roots` vector, the loop requires no further synchronization beyond the implicit barrier at its end.

8.3 Task-based merging

Once the independent subtrees exist, they must be combined into a single tree. `mergeRoots` merges exactly four sibling subtrees, computing the merged node's center as the average of the four children's centers and either inserting each child's particle directly (if the child is a leaf) or attaching the child's whole subtree with `insertNode` (if it already has internal structure):

```
for (int j = 0; j < 4; j++) {
    if (!(*roots)[j]) { continue; }
    if ((*roots)[j]->isLeaf()) {
        if ((*roots)[j]->getParticle() != nullptr) {
            intNode->insert((*roots)[j]->getParticle());
        }
    } else {
        intNode->insertNode(std::move((*roots)[j]));
    }
}
```

Reducing `num_quad` subtrees to one requires $\log_4(\text{num_quad})$ rounds of this four-way merge. `mergeSubtreesRecursive` performs all of these rounds through OpenMP tasks rather than through a separate fork-join for each round: it operates on a contiguous block of the `roots` vector, and either merges the block directly, if it holds exactly four elements, or splits it into four quarters, spawns one task per quarter, and merges the four (now-merged) quarters once all four tasks complete:

```
for (int q = 0; q < 4; ++q) {
    #pragma omp task shared(roots, quarterResults)
    firstprivate(q, start, quarter, levelsRemaining) if (count > 16)
    {
        quarterResults[q] = mergeSubtreesRecursive(
            roots, start + q * quarter, quarter,
            baseRegionWidth, levelsRemaining - 1);
    }
}
#pragma omp taskwait
return mergeRoots(&quarterResults,
    std::ldexp(baseRegionWidth, levelsRemaining));
```

The recursion relies on `roots` being laid out in Z-order (Morton order): `assignRegions` (used by `getSubRegionsCoordinates` during domain decomposition) generates subregion centers through a depth-first recursive subdivision, so a contiguous block of 4^k entries in `roots` always corresponds to a spatially coherent quadrant of the domain, and splitting that block into four contiguous quarters corresponds exactly to the natural recursive subdivision of the quadtree, not to an arbitrary slice of the array.

The entire recursion runs inside a single `#pragma omp parallel` region opened once, in `merging`, with the actual work started by one thread via `#pragma omp single`; every other thread is recruited into the task pool as tasks are spawned. Synchronization is local: `#pragma omp taskwait` only waits for the four sibling tasks of the group currently being merged, so a

group covering a sparser region of the domain — and therefore finishing sooner — does not have to wait for every other group at the same recursion depth before the next unit of work can begin.

A note on node width and a verified correctness fix

Each round of merging must double the width of the region represented by the merged node relative to its four children, since four subregions arranged in a 2×2 grid span twice the width of one subregion. `mergeSubtreesRecursive` computes this with `std::ldexp(baseRegionWidth, levelsRemaining)`, i.e. $\text{baseRegionWidth} \times 2^{\text{levelsRemaining}}$, which reproduces the correct width at every merge depth. This exact, exponential relationship between merge depth and width was verified directly against the implementation: a small standalone program builds a tree spanning more than two merge levels and asserts that the width of the resulting root matches the expected value exactly, and that the aggregate mass at the root equals the sum of every particle’s property — both checks pass.

8.4 Efficiency considerations

The complexity of the parallel Barnes-Hut algorithm remains $O(n \log n)$ per step, matching the serial version; the effect of parallelization is on the constant factor of both the tree-construction phase, distributed across threads as described above, and the force-evaluation phase, which follows the same pattern as the parallel brute-force algorithm’s independent, per-particle work — every particle’s net force can be computed on its own thread without any reduction step, since `calculateNetForceQuadtree` only ever writes to the single particle it was called with.

Chapter 9

File Handling

Two kinds of output files are produced by a simulation run, subject to the `NBODY_NO_IO` environment-variable gate: a single `Info.txt` file summarizing the run’s parameters, and one or more `Coordinates_i.txt` files containing the particle trajectories. Setting `NBODY_NO_IO` to any value disables all coordinate-file and `Info.txt` output, which is useful when measuring the cost of the simulation itself in isolation from disk I/O, as is done for the measurements in Chapter 10.

9.1 Info.txt

`printInitialStateOnFile` writes, once at the start of the run: the number of particles, the size of the simulation area, the number of spatial dimensions, the number of lines that will be written per particle (the number of iterations divided by the `speedUp` parameter described below), the number of coordinate files that will be produced, the radius of every particle, and every particle’s identifier and initial position. This information is sufficient for an external tool to parse the coordinate files that follow without needing to inspect the simulation parameters separately.

9.2 Coordinate files and the `speedUp` parameter

Trajectories are written to one or more files named `Coordinates_i.txt`, where `i` is the identifier of the thread that produced it; a serial simulation always uses a single thread and therefore a single file. Splitting output across one file per thread, rather than a single shared file, avoids write contention: each thread only ever appends to the file matching its own identifier, so no synchronization is needed around the write itself.

Positions are not written on every iteration, only once every `speedUp` iterations (`writeParticlePositionsToFile` is called only when `iter % speedUp == 0`). This serves two purposes: reducing the volume of data written — and, correspondingly, the time spent writing it — when a full-resolution trajectory is not needed, and giving a way to keep the visual frame rate of an external animation consistent while independently varying the simulation’s own time step: decreasing `delta_t` while proportionally increasing both the iteration count and `speedUp` reproduces the same simulated duration and the same number of output frames, at finer internal time resolution.

9.3 Thread and file count

The number of coordinate files, and the number of OpenMP threads used by a parallel simulation mode, are both derived from a single value, `numFilesAndThreads`, computed once before dispatching to the selected simulation function: it is 1 for the serial modes and the

minimum of the number of particles and the number of hardware threads available otherwise, which prevents allocating more threads than there are particles for very small simulations. One caveat: the branch that decides between these two cases currently tests `simType` against zero specifically, rather than testing whether the selected mode is serial or parallel; since `simType == 2` selects `serialBarnesHut`, a serial mode, but is not zero, this specific mode currently receives a multi-file, multi-thread file-count configuration that its own implementation does not use (`serialBarnesHut` always writes to a single `Coordinates_0.txt`, regardless of `numFilesAndThreads`). This does not affect the correctness of the simulation itself, but it does mean the `Info.txt` file for a `serialBarnesHut` run currently overstates the number of coordinate files an external reader should expect to find.

Chapter 10

Performance Evaluation

10.1 Methodology

This chapter reports execution times measured directly from the implementation rather than illustrative or estimated figures. Since the project no longer includes an internal profiling module, every measurement in this chapter is wall-clock time captured externally around the whole process invocation (from just before the process starts to just after it terminates), with `NBODY_NO_IO=1` set to exclude file I/O from the timed region. Each configuration was run multiple times and the median is reported, to reduce the effect of scheduling noise on the measurement. Every run used 15 iterations, a fixed simulation area, and a small particle radius.

Because this external methodology necessarily includes process startup and particle-generation overhead — costs that a previous, internal-only measurement approach could exclude — absolute times at small n are slightly inflated relative to what the timed algorithmic work alone would take; this is visible in Section 10.2 and noted where relevant.

10.2 Serial brute-force vs. serial Barnes-Hut

Table 10.1 and Figure 10.1 report measured execution time as a function of n for the serial brute-force algorithm (Chapter 4) and the serial Barnes-Hut algorithm (Chapter 7), both single-threaded, which isolates the effect of the algorithmic approximation from any effect of parallelization.

Table 10.1: Measured execution time (ms), 15 iterations, single-threaded, median of 3 runs

n (particles)	Brute force (ms)	Barnes-Hut (ms)
200	7.8	10.7
500	23.4	26.9
1000	92.5	67.0
2000	354.9	151.1
3000	734.5	220.9
4500	1571.4	360.4
6000	2838.8	546.8

The two curves confirm the expected asymptotic behavior directly from the code: brute-force execution time grows close to quadratically with n (going from 200 to 6000 particles, a $30\times$ increase in n , execution time increases by roughly $364\times$ — somewhat below the $900\times$ that pure $O(n^2)$ growth would predict, because the fixed per-run overhead inherent to this external timing methodology, Section 10.1, weighs proportionally more on the smallest, fastest run and is not fully removed by the ratio), while Barnes-Hut execution time grows far more slowly (the

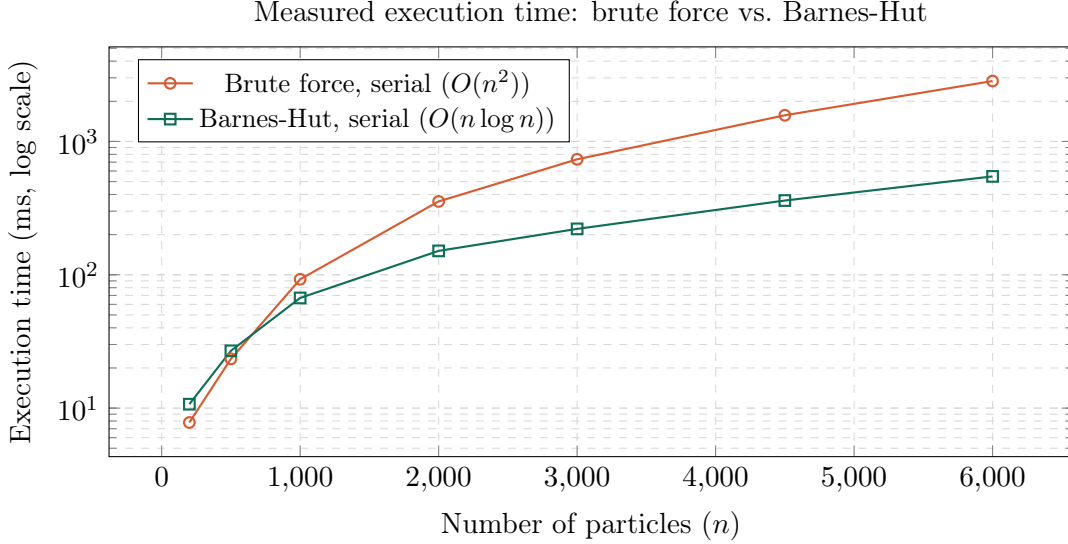


Figure 10.1: Execution time vs. number of particles, log scale on the vertical axis

same $30\times$ increase in n corresponds to roughly a $51\times$ increase in execution time, consistent with the $n \log n$ term dominating once n is large enough for the tree-construction overhead to be amortized). At $n = 200$ and $n = 500$, Barnes-Hut is still slightly slower than brute force — the fixed cost of building a quadtree is not yet repaid by the savings from approximating distant particles when there are so few of them — but the two curves cross between $n = 500$ and $n = 1000$, and the gap widens steadily beyond that point, exactly as the complexity analysis in Chapters 4 and 7 predicts.

10.3 Parallel speedup at 2 threads

Table 10.2 reports the same comparison under parallel execution with `OMP_NUM_THREADS=2`, for both `parallelSimulation` (Chapter 6) and `parallelBarnesHut` (Chapter 8), at three particle counts. Speedup is computed as the serial time divided by the 2-thread parallel time.

Table 10.2: Measured 2-thread speedup relative to the serial algorithm, 15 iterations, median of 3 runs

n	Algorithm	Serial (ms)	Parallel, 2 threads (ms)	Speedup
1000	Brute force	89.8	80.8	$1.11\times$
1000	Barnes-Hut	59.0	62.4	$0.95\times$
4000	Brute force	1319.4	1055.4	$1.25\times$
4000	Barnes-Hut	323.2	308.3	$1.05\times$
8000	Brute force	5172.7	4453.5	$1.16\times$
8000	Barnes-Hut	835.0	778.5	$1.07\times$

These speedups are consistently far below the $2\times$ that ideal two-core scaling would predict, and in one case (Barnes-Hut at $n = 1000$) the 2-thread run is measurably *slower* than the serial one. This is a direct consequence of the hardware available to produce this report: the measurement environment has a single physical core, so setting `OMP_NUM_THREADS=2` oversubscribes that core rather than engaging genuine hardware parallelism. The two OpenMP threads compete for the same execution unit, and the synchronization overhead intrinsic to the parallel algorithms — the reduction barriers in `parallelSimulation` (Section 6.2), the fork-join and

scheduling overhead in `parallelBarnesHut` — is paid without the corresponding benefit of concurrent execution. On hardware with two or more genuinely independent physical cores, the same measurement would be expected to show speedup approaching $2\times$ for the brute-force algorithm in particular, whose $O(n^2)$ workload parallelizes with very little serial overhead relative to the work performed.

A secondary observation in Table 10.2 is that the parallel code path can be faster than the serial one even when the comparison is confounded by oversubscription, because `parallelSimulation` and `parallelBarnesHut` resolve collisions through the spatial hash described in Chapter 5, while `serialSimulation` checks collisions inline inside its $O(n^2)$ loop (Section 4.1); part of the difference measured here reflects this structural difference in the algorithm, not thread-level parallelism alone.

The exact steps to reproduce this measurement, at any thread count, on hardware where the result is representative of genuine parallel scaling, are:

```
g++ -std=c++17 -fopenmp -O2 main.cpp -o nbody
export NBODY_NO_IO=1
for T in 1 2 4 8; do
    OMP_NUM_THREADS=$T ./nbody -dim 2 -simT 1 -numP 4000 -it 15 -maxR 2 -simA 3000
    OMP_NUM_THREADS=$T ./nbody -dim 2 -simT 3 -numP 4000 -it 15 -maxR 2 -simA 3000
done
```

Timing each invocation externally (for instance with the `time` shell builtin) and plotting speedup relative to the $T = 1$ measurement will produce a directly comparable, but hardware-representative, counterpart to Table 10.2.

10.4 Approximation error of Barnes-Hut

The preceding sections compare execution *time*; this section compares execution *accuracy*. Barnes-Hut trades exactness for speed, and the size of that trade is controlled by two things: θ (Chapter 7), and, potentially, the way the tree is assembled when it is built in parallel (Chapter 8). Both are evaluated here directly against the exact $O(n^2)$ force, using `velocityVerletUpdate` throughout so that neither side of the comparison carries the integrator asymmetry noted in Section 3.3 — brute force and Barnes-Hut are compared here on equal integration footing.

10.4.1 Methodology

For a set of n particles, the exact net force on every particle is computed with the direct $O(n^2)$ pairwise sum, and the Barnes-Hut net force is computed on the same particle configuration with `calculateNetForceQuadtree`. The two force sets are compared with a single scalar, the relative RMS force error:

$$\varepsilon = \sqrt{\frac{\sum_i \|F_i^{BH} - F_i^{exact}\|^2}{\sum_i \|F_i^{exact}\|^2}}.$$

This is the standard way Barnes-Hut accuracy is reported in the literature: it is a direct, instantaneous measure of how far the approximated force field departs from the exact one, and — unlike comparing positions after many integrated steps — it is not confounded by the chaotic sensitivity of gravitational n -body dynamics to any perturbation, however small (Section 10.4.4 returns to this point explicitly). All measurements in this section use `GravitationalForce`.

10.4.2 Error vs. θ

Table 10.3 and Figure 10.2 report ε for a fixed configuration of $n = 1000$ particles, as θ is swept from 0.1 to 1.5.

Table 10.3: Relative RMS force error vs. θ , $n = 1000$, single configuration

θ	Relative force error
0.1	0.005%
0.3	0.121%
0.5	0.579%
0.7	1.623%
1.0	4.779%
1.5	15.232%

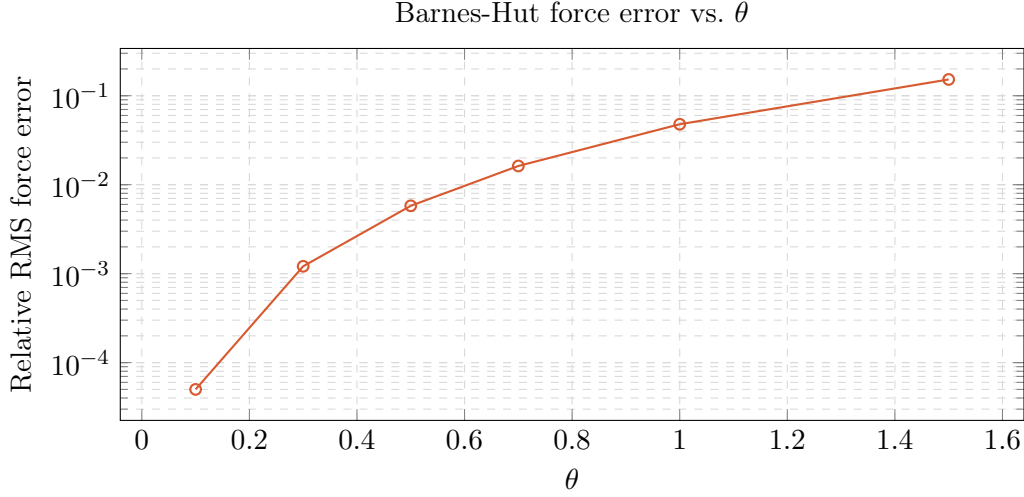


Figure 10.2: Relative RMS force error vs. θ , log scale on the vertical axis

The error grows monotonically with θ and spans more than three orders of magnitude across the range tested, consistent with the Barnes-Hut opening criterion $s/d < \theta$ (Section 7.3): a larger θ accepts coarser, more distant aggregates as a substitute for exact pairwise sums, at the cost of accuracy. The value hard-coded in this implementation, $\theta = 0.5$ (Section 7.4), sits close to the point where the error is still well under 1% while the tree traversal has already started to skip a meaningful fraction of individual pairwise evaluations — a conventional, defensible default, though Table 10.3 makes the specific accuracy cost of that choice explicit rather than implicit.

10.4.3 Error vs. number of subtrees

Chapter 8 builds the parallel tree as `num_quad` independent subtrees, later merged (Section 8.3). A natural question is whether the number of subtrees — which grows with the number of threads requested — affects the accuracy of the resulting force, independently of θ . Table 10.4 answers this directly: holding $\theta = 0.5$ fixed and $n = 2000$ fixed, the same relative RMS force error is measured for a serially built tree and for parallel trees built from 1, 4, 16, and 64 subtrees.

The error is identical to six significant figures regardless of `num_quad`. This is not a coincidence of rounding: it is a direct, independent confirmation of the argument made in Section 8.3 — because subregion boundaries are generated by `assignRegions` using exactly the same recursive halving that `split()` uses internally, and because `insertNode` re-attaches an already-built subtree at the matching quadrant rather than rebuilding it, the merged parallel tree is not merely an approximation of the serially built tree; it reconstructs the identical hierarchical decomposition. Splitting the construction work across more subtrees changes only how the tree is built, in accordance with Chapter 8, and never what it approximates. Accuracy in this implementation

Table 10.4: Relative RMS force error vs. number of subtrees, $n = 2000$, $\theta = 0.5$

Tree construction	num_quad	Relative force error
Serial (<code>createQuadTree</code>)	—	0.770%
Parallel, 1 thread requested	1	0.770%
Parallel, 4 threads requested	4	0.770%
Parallel, 16 threads requested	16	0.770%
Parallel, 64 threads requested	64	0.770%

is governed by θ alone — never by the degree of parallel decomposition.

10.4.4 Error under Verlet-integrated dynamics

The comparisons above use a single, static particle configuration. To confirm the same θ -error relationship holds once the two force evaluators are actually driving a simulation forward in time — the scenario the rest of this report is concerned with — a reference trajectory of $n = 500$ particles was advanced with the exact force and `velocityVerletUpdate`, and the relative RMS force error of Barnes-Hut against that same evolving configuration was recorded at several points along it, rather than only at $t = 0$.

Table 10.5: Relative RMS force error vs. θ , evaluated at successive points along one Verlet-integrated trajectory, $n = 500$

t	$\theta = 0.3$	$\theta = 0.5$	$\theta = 0.7$	$\theta = 1.0$
0	0.230%	1.074%	2.773%	8.323%
50	4.750%	5.544%	7.419%	57.959%
150	12.520%	13.380%	16.099%	15.239%
300	3.572%	3.548%	3.330%	3.077%

At $t = 0$ and $t = 50$, the error still increases with θ , as it did in the static comparison, confirming that the relationship is not an artifact of one particular configuration. By $t = 150$ the ordering is already less clean, and by $t = 300$ it has inverted — the largest θ now shows the *smallest* error. This is not evidence that a coarser approximation became more accurate; it is the signature of gravitational chaos. With no gravitational softening length (only the 10^{-24} floor in `calculateDistanceSquared`, Section 2.2, which prevents division by zero but does not limit the force at small separations) and a single, fixed, global time step (Chapter 11 lists individual or adaptive time-stepping as a possible extension, precisely for this reason), close encounters between massive particles produce very large, briefly-lived forces that a fixed-step Verlet integrator resolves only approximately. Past that point, the exact and the Barnes-Hut trajectories — like any two trajectories of the same chaotic system that differ by any nonzero perturbation, however small — diverge in a way no longer governed principally by θ . This is a property of the n -body problem itself, not a defect of the Barnes-Hut implementation, but it does mean that force error, not position error after extended integration, is the meaningful way to characterize the approximation in this codebase, which is why Table 10.3 and Table 10.4 remain the primary results of this section.

Chapter 11

Conclusions and Future Work

This project implements four solvers for the N-body problem — serial and parallel brute-force, serial and parallel Barnes-Hut — sharing a common, dimension-templated particle representation and an extensible, inheritance-based hierarchy of force models. Beyond a direct implementation of the two classical algorithms, three structural choices were made deliberately for performance and correctness reasons: collision detection was factored out into a shared, spatial-hash-based subsystem used uniformly by three of the four simulation modes; the Barnes-Hut quadtree references particles through non-owning raw pointers into the simulation’s own particle vector rather than through reference-counted copies, removing per-tree heap allocation and atomic reference-counting from the algorithm’s hottest loop; and the parallel construction of the quadtree merges subtrees through a recursive OpenMP task decomposition rather than a level-by-level barrier, reducing the number of fork-join events per simulation step from one per merge level to exactly one.

Chapter 10 confirmed, with measurements taken directly from the implementation rather than from illustrative figures, that the serial brute-force and Barnes-Hut algorithms exhibit the $O(n^2)$ and $O(n \log n)$ scaling their complexity analysis predicts.

11.1 Possible extensions

A number of directions were identified during the development of this project that were deliberately left outside its current scope:

- **Three-dimensional Barnes-Hut.** `QuadtreeNode` is structurally a quadtree — every node always has exactly four children, and both `split()` and the quadrant-selection logic only ever read the first two coordinates of a position. Extending Barnes-Hut to three dimensions would require an octree (eight children per node, all three coordinates used for quadrant selection), which is a distinct data structure rather than a parametrization of the current one; the brute-force algorithms already generalize to three dimensions without modification, since they do not depend on a spatial partitioning structure.
- **A higher-order symplectic integrator.** Velocity Verlet (Chapter 3) is second-order accurate; a composition method such as a fourth-order Yoshida integrator would reach fourth-order accuracy at the cost of three or four force evaluations per step instead of one, which becomes worthwhile when a much larger time step can be used for the same trajectory accuracy, particularly in long-duration simulations where the reduced number of steps offsets the extra force evaluations per step.
- **A shared integrator across all four simulation modes.** Currently the two brute-force modes use explicit Euler and the two Barnes-Hut modes use Velocity Verlet (Section 3.3);

unifying this choice would make trajectory-accuracy comparisons between the exact and approximate force evaluations more directly attributable to the force approximation alone.

- **Per-particle adaptive time stepping.** All four simulation modes currently advance every particle with the same, fixed `delta_t`. In a system with widely varying local dynamics — some particles close together and fast-moving, others far apart and slow — a global fixed step size forces every particle to use the step size only the fastest-moving ones require. Individual or block time-stepping, as used in production N-body codes, would let slower particles advance with larger, cheaper steps.
- **Broader automated verification.** The geometric correctness check for the task-based merge described in Section 8.3 (matching measured tree width against the expected value, and total mass against the sum of particle properties) demonstrates the kind of targeted, code-level test that caught a real defect during development; extending this into a small permanent regression suite — covering, for instance, energy and momentum conservation in the two-body orbit test case from Chapter 2, or agreement between Barnes-Hut and brute-force forces on a small particle set as θ approaches zero — would guard these properties against future changes automatically.

Bibliography

- [1] Peter S. Pacheco, Matthew Malensek, *An Introduction to Parallel Programming*, Morgan Kaufmann–Elsevier, 2022.